Navigating the Collector

A Contributor's Field Guide

Kemal Akkoyun · OpenTelemetry Maintainer · Datadog

Why Does a PR Take Three Weeks?

Ondrej Balaz's mdatagen PR — 452 additions, eight comments, merged in three weeks:

```
May 07  ✓ CLA signed
May 07  → "should I also implement multipleOf?"
May 08  ← "no use case yet, wait"
May 09  → "linter pass — missed make fmt"
May 11  ← "please rebase and fix CI failures"
May 12  → "done, can you re-run the CI?"
May 15  → "can you help me start the CI workflows?"
May 26  ✓ merged
```

This is **not** a Bloomberg problem. It is the ecosystem.

opentelemetry-collector#15270

Three Axes of Complexity

Architectural	Social	Historical
multi-module repo	CODEOWNERS sprawl	renamed types
`go.mod` per component	maintainer math	moved packages
codegen / mdatagen	review SLOs	dead tutorials
stability tiers	stale-bot churn	accreted APIs

The Collector is complicated for structural reasons, not accidental ones.

> The way through is to work *with* the constraints — not around them.

Why Is It Complicated?

Architectural

Two Repositories. Two Module Structures.

opentelemetry-collector (core) — ~25 module groups in `versions.yaml`, two versioning tracks (stable / beta).

opentelemetry-collector-contrib — 300+ independent `go.mod` files, one per component:

```
receiver/hostmetricsreceiver/go.mod
receiver/kafkareceiver/go.mod
processor/attributesprocessor/go.mod
extension/basicauthextension/go.mod
...
```

Why? **Version isolation.** A security fix in one receiver's dependency should not force a version bump across unrelated components.

The cost: **every component is an independent module release.**

versions.yaml · 0093005 — core's module groups

The Module Version Map

`versions.yaml` defines release sets — two parallel versioning tracks:

```yaml
module-sets:
  stable:
    version: v1.59.0      # ← semver stability guaranteed
    modules:
      - go.opentelemetry.io/collector/pdata
      - go.opentelemetry.io/collector/component
      - go.opentelemetry.io/collector/confmap
      - go.opentelemetry.io/collector/pipeline
      ...
  beta:
    version: v0.153.0     # ← may break between minor releases
    modules:
      - go.opentelemetry.io/collector
      - go.opentelemetry.io/collector/internal/memorylimiter
      ...
```

Stable APIs move slower. Beta can break. Know which track you are on.

The Lockstep Problem

Lockstep: two systems that can only advance in sync — neither can release independently of the other.

Change a core interface → must update every contrib module that depends on it.

```
opentelemetry-collector (core)          opentelemetry-collector-contrib
──────────────────────────────          ──────────────────────────────

1. PR-A: change API
2. cut release  ──────────────────►     (waits for release tag)
                                        3. PR-B: bump go.mod
                                             replace directives
                                             in N component modules
```

A **single API change** requires coordinated PRs across two repositories,
N `go.mod` files, weeks of stagger.

This is why large refactors are rare and slow.
Scope your work to stay within one module boundary.

Codegen Is the Source of Truth

Every component has a `metadata.yaml` — the single source of truth for identity, stability, owners, and feature gates:

```yaml
# receiver/hostmetricsreceiver/metadata.yaml
type: host_metrics
status:
  class: receiver
  stability:
    beta:        [metrics]
    development: [logs]
  codeowners:
    active: [dmitryax, braydonk, rogercoll]
feature_gates:
  - id: receiver.hostmetricsreceiver.UseLinuxMemAvailable
    stage: beta
```

receiver/hostmetricsreceiver/metadata.yaml · 3c4f53a

Codegen Is the Source of Truth

`mdatagen` reads `metadata.yaml` and generates Go code you never edit by hand:

```
metadata.yaml  →  internal/metadata/generated_metrics.go
               →  internal/metadata/generated_config.go
               →  internal/metadata/generated_status.go
```

Edit `metadata.yaml`. Never edit the generated files.

The feature-gate migration PRs are doing exactly this — moving feature gate declarations from hand-written Go into `metadata.yaml` so `mdatagen` owns the output.

Stability Tiers Set Expectations

Tier	Meaning	API guarantees	Review pace
Development	experimental	none	slow / may not merge
Alpha	usable, not stable	breaking ok	moderate
Beta	stable API intended	no breaks	moderate–fast
Stable	production-grade	no breaks	fast (high scrutiny)
Deprecated	scheduled removal	—	—

`hostmetricsreceiver`**: beta for metrics, development for logs —** same component, different stability per signal type.

Know the tier before you PR. It sets realistic expectations.

Why Is It Complicated?

Social

CODEOWNERS Is the Org Chart

Each component declares its owners in `metadata.yaml`:

```yaml
# receiver/hostmetricsreceiver/metadata.yaml
codeowners:
  active: [dmitryax, braydonk, rogercoll]
```

When you open a PR, GitHub routes review to those humans —
across multiple companies, time zones, day jobs.

```
  Your PR
    │
    └──► GitHub assigns → dmitryax  (Datadog,  UTC−8)
                          braydonk  (Google,   UTC−5)
                          rogercoll (Grafana,  UTC+2)
```

Review latency is not indifference. It is calendar math.

Maintainer Math

250+ components in contrib. A handful of core maintainers who can approve anything. Component approvers who can only approve their slice.

The "priority components" list exists because maintainers had to triage **themselves.**

~30–50 PRs/week across all of contrib. Finite reviewers.

	Review wait
Non-priority component	days to weeks
Priority component	1–3 days

This is not a broken process. It is a governance model under load.

Find the priority list: contrib#44130

The Welcome Bot Encodes the Culture

Every first-time contributor PR to contrib gets this — **verbatim:**

> First-time contributors should have **at most one PR not marked as draft** until their first PR is merged.
>
> If your change isn't one of our **priority components**, reviews may take more time.
>
> Give reviewers **at least a few days** before pinging them for feedback.
>
> If you need help... raise the topic on **#otel-collector-dev** or a **Collector SIG meeting.**

The rules are already written down. **Read them.**

opentelemetry-collector-contrib#48716 · first-time contributor welcome

Stale-Bot Is a Tool, Not a Verdict

First Response

~3,000 open issues in contrib. Without the stale-bot, noise drowns signal.

Issues and PRs inactive for **60 days** → `stale` label. After 30 more days → **auto-close.**

When your PR goes quiet for 7 days:

- Rebase on `main`
- Ping the CODEOWNER — once, politely, with context

Stale-Bot Is a Tool, Not a Verdict

Escalation Path

Still quiet after 30 days:

- Raise in `#otel-collector-dev`
- Add to Collector SIG meeting agenda
- Two full cycles with no movement? **Come back with a smaller scope.**
- An 8-month gap before a codeowner replied is real (contrib#35305). **Plan for it.**

Why Is It Complicated?

Historical

Old PRs Lie

The Collector has 4+ years of active evolution. A PR from 2022 is not wrong — it was correct for its version. But it will confuse you today.

Change	Old pattern	New pattern	Since
Data types	`model/pdata.AttributeMap`	`pcommon.Map`	v0.49.0 · Apr 2022
Config loading	`config.MapProvider`	`confmap.Provider`	v0.53.0 · Jun 2022
Component type	`component.Type("foo")`	`component.MustNewType("foo")`	v0.97.0 · Feb 2024
Scraper helper	`receiver/scraperhelper`	`scraper/scraperhelper`	v0.117.0 · Jan 2025

Before opening an issue or following an example: check when it was written.

The Pattern That No Longer Compiles

Every component factory changed in **February 2024** (v0.97.0):

```
// Pre-v0.97 — every tutorial and blog post before Feb 2024 still shows this
const typeStr = component.Type("hostmetrics")

// Post-v0.97 — required today; bare string is a compile error
var typeStr = component.MustNewType("hostmetrics")
```

`MustNewType` validates the string at startup (ASCII alphanumeric + `_`, starts with a letter). The error message does not say "use MustNewType" — it says the type doesn't exist.

This affects **every receiver, processor, exporter, connector, and extension factory.**

collector#9414 · v0.97.0 · Feb 2024

When the Import Path Moved: confmap

Config loading was extracted from `service` into its own module in Jun 2022 (v0.53.0):

```
// Pre-v0.53 — visible in any PR or doc from before mid-2022
import "go.opentelemetry.io/collector/config"
config.MapProvider, config.Map

// Now
import "go.opentelemetry.io/collector/confmap"
confmap.Provider, confmap.Conf
```

Pavel's confmap work (#48414) builds directly on this module.
Old tutorials that show `config.Map` are not wrong — they predate the extraction.

collector#5237 · v0.53.0 · Jun 2022

When the Import Path Moved: scraper

`scraperhelper` was moved out of the `receiver` module in Jan 2025 (v0.117.0):

```
// Pre-v0.117 — still the import path in most contrib components today
import "go.opentelemetry.io/collector/receiver/scraperhelper"

// Now
import "go.opentelemetry.io/collector/scraper/scraperhelper"
```

This is recent enough that most open PRs and the majority of contrib components still use the old path. Expect to see both in the wild.

Rule: if a PR's imports don't resolve, look up the version boundary before rewriting.

collector#11985 · v0.117.0 · Jan 2025

Deciphering Convoluted Issues

Read the Labels First

GitHub labels tell you the invitation level before reading a single line of the body:

Label	What it signals
`good first issue`	Bounded scope, codeowner-tagged, clear acceptance
`help wanted`	Maintainers want this but don't have bandwidth
`needs triage`	No owner yet — lower urgency
`priority:p1` / `p2`	Actively tracked
`lifecycle:unmaintained`	No active owners — may not merge
`area:confmap` / `receiver/`	Routes to subsystem or component

Labels encode the invitation level. Start there.

Trace Config → Component → Owner

Given any pipeline fragment — here is the path to the right human:

```
receivers:
  hostmetrics:
    collection_interval: 10s
```

1. `"hostmetrics"` → `receiver/hostmetricsreceiver/`
2. Open `metadata.yaml` → read `status.stability` and `codeowners.active`
3. Check last merged PR by those handles — active recently?
4. Open an issue: describe the problem, not the solution. Ping the owner.

You can do this for any component in the repo.

Recognize a Good First Issue in the Wild

[internal/aws] migrate to AWS v2 SDK · contrib#37728

Scope	One package — bounded, no cross-module blast
Owner	@mx-psi actively engaged in umbrella #36699
Shape	"remove v1 imports, add v2 equivalents" — clear acceptance criteria
Tier	Not stable — API changes allowed

Recognize a Good First Issue in the Wild

[cmd/mdatagen] Move feature gates to metadata.yaml · #46116

Scope	One component at a time — take as many as you want
Shape	Mechanical migration — hard to get catastrophically wrong
Pattern	Every feature gate needs this — a stack of work is available

The shape of a good entry door: bounded, owner-engaged, clear acceptance.

The Playbook

Signal Intent First

Signal Intent First

The single most important thing I want you to take from this session:

- **Ask before building.**
- **Build the smallest thing that proves the idea.**
- **Make it easy for the maintainer to say yes.**

Signal Intent First

The Why

A PR that surprises a maintainer is one they must evaluate from scratch.

A PR that follows a tracked issue and a prior conversation
is one they can approve in 10 minutes.

The work is often the same. The outcome is not.

Four Questions Before You Write Code

Before opening a PR — or writing a single line:

- **Is this a priority component?**
  → contrib#44130
- **Has this been discussed before?**
  → Search closed PRs *and* SIG meeting notes — closed ≠ rejected
- **Who are the CODEOWNERS and are they active recently?**
  → `metadata.yaml` → `codeowners.active` → check their last merged PR
- **Should I open an issue first?**
  → "I'd like to implement X because Y. Does this direction make sense?"

Where to Signal

GitHub issue
Use when: new idea, feature request, bug report
Do: describe the *problem*, not the solution
Link: related issues, prior art, your use case

Draft PR
Use when: you have a prototype worth discussing
Do: open as draft, mark "not for review yet"
Signals: you're serious, here's the shape of my thinking

#otel-collector-dev (CNCF Slack)
Use when: quick sanity-check, "is this the right approach?"
Do: one message with context + link to your issue

Collector SIG meeting
Use when: blocked PR, cross-cutting RFC, design question
Do: add to agenda 48h in advance
→ SIG calendar

Case Study A: The Secret Manager Saga

The problem — Thomas's RFC (contrib#48277):

> "A lot of production infrastructure stores and rotates secrets through managed stores. The gap means teams end up **running a sidecar just to fetch the secret and write it to disk** so the file watcher can pick it up."
>
> Happy to contribute this upstream if there's appetite for it.

Case Study A: What Happened

- Pavel opens #48380 — full AWS impl inside `basicauthextension`
- Thomas: "*separate ext avoids pulling AWS SDK into basicauthextension*"
- Pavel closes #48380, opens #48414 — extract shared logic → `extension/internal/basicauth` **(MERGED)**
- Larry comments on #48277 RFC *before* writing code — signals intent, then opens #48484
- Thomas → #48406 (AWS extension)
- Mikiyas → #48474 + #48477 (AWS + GCP extensions)
- Larry → #48484 (Azure Key Vault extension)

Case Study A: The Lesson

Four engineers. Four cloud providers. Four parallel PRs. All mergeable.

Without early coordination

- 4 PRs duplicating basicauth logic
- each pulling a different cloud SDK
- maintainer: dependency graph wrong, can't accept

With early coordination

- 1 shared internal pkg (Pavel) — MERGED
- 3 cloud-specific extensions, each reviewable alone
- clean dependency graph, maintainer says yes

Pavel: "*Yes, a shared internal module sounds like the right approach.*"
Thomas: "*thanks! We'll update our PR to be built on top of your PR changes now!*"

contrib#48277 · contrib#48414 · contrib#48474 · contrib#48484

Case Study B: The 7-Line PR

Abhimanyu's depguard rule (contrib#48716):

```
+          aws-sdk-v1:
+            deny:
+              - pkg: github.com/aws/aws-sdk-go
+                desc: "Use github.com/aws/aws-sdk-go-v2 instead. See #36699"
```

The PR description opens with:

> "Per discussion with @mx-psi in #36699, this is the **recommended first step** to prevent regression while remaining component migrations complete."

Seven lines. One sentence linking to the prior conversation. **Immediately reviewable.**

Case Study B: The 7-Line PR

The Lesson

Contrast: a 500-line cross-component migration, no prior discussion, lands cold in a maintainer's queue → weeks of back-and-forth, possible rejection.

Small PRs with prior context are the fast lane.

contrib#48716 · contrib#36699

Case Study C: The Cold Landing

Ondrej Balaz's mdatagen PR — same one as the opening slide. No prior issue, no RFC, code landed cold:

```
May 07  CLA bot fires
May 07  Ondrej:      "should I also implement multipleOf?"   ← scope question  ✓
May 08  maintainer:  "no use case yet, don't bother"         ← scope locked, proceed
May 09  Ondrej:      "linter pass, missed make fmt"            ← linter bite
May 11  maintainer:  "please rebase and fix CI failures"
May 12  Ondrej:      "done — can you re-run the CI?"         ← CI needs maintainer
May 15  Ondrej:      "can you help me start the CI workflows?"
May 26  merged
```

Three weeks. 452 additions. **Not unusual. Not a failure.**

The scope question on day one was exactly right —
it saved a revision cycle later.

opentelemetry-collector#15270

The Survival Moves: During a PR

Situation	Action
CLA bot fires	Sign once at identity.linuxfoundation.org — never again
CI needs a re-run	Comment: `@<codeowner> could you trigger CI?`
"rebase" request	`git fetch upstream && git rebase upstream/main` — do it same day
Linter failure	`make fmt lint` before every push

The Survival Moves: When It Goes Quiet

Situation	Action
No response 7 days	One polite ping to the CODEOWNER named in `metadata.yaml`
No response 30 days	Raise in `#otel-collector-dev` or Collector SIG agenda
Stale-bot fires	Comment to reset; rebase if behind
Two cycles, still stuck	Reconsider scope — is there a smaller first step?

One PR at a time until your first merge. The welcome-bot rule exists for a reason.

What Contributions Are Most Welcome Right Now

Contribution type	Why it moves fast
Bug fixes on priority components	Fast review lane
Component README / doc gaps	Low risk, always welcome
`metadata.yaml` feature-gate migrations	Mechanical · 40-line PRs like Pavel's #48619 · umbrella: #46116
Semconv attribute updates	Contributor-friendly, active community
AWS SDK v1 → v2 migration	Active umbrella #36699, owner engaged

Before You Start

Check three things before writing a line of code:

1. Is it on the priority list? → contrib#44130
2. Does an umbrella issue exist? → claim it, don't create a parallel one
3. Is the codeowner active? → check their last merged PR in that component
4. **Avoid as a first PR:** new component from scratch, cross-repo API changes, anything touching `pdata` or `confmap` core internals.

Closing

The Maintainer's Plea

Speaking as one:

- **Small PRs.** One idea. One module. Reviewable in one sitting.
- **Intent first.** Open an issue. Start a thread. "*Does this direction make sense?*" before "*Here is my 800-line PR.*"
- **Patience with pings.** A few days. Not a few hours. We are not ignoring you.

Your Next Step Tonight

Pick one component you have used or are working on.

1. Find `receiver/<name>/metadata.yaml` or `extension/<name>/metadata.yaml`
2. Read `status.stability` + `codeowners.active`
3. Search open issues labeled `good first issue` scoped to that component
4. Comment: "*I'd like to take a look at this.*"

Your Next Step Tonight

That comment tells the codeowner a human is alive and interested. It starts the conversation before the code exists.

The contribution starts before the code.

References

Your work — keep going:

- Secret-manager cluster: contrib#48277
- AWS SDK v2 migration: contrib#36699
- Priority components: contrib#44130

Community:

- #otel-collector-dev (CNCF Slack)
- Collector SIG meeting
- CONTRIBUTING.md
- mdatagen README

Curated issues (mentorship spreadsheet):

docs.google.com/spreadsheets/d/1XVU-KC438q4hEr6Mx1YZD4LdGGt5FXGSSZUbB3_kSrY

Reading:

- Bloomberg: Sustaining OpenTelemetry
- CNCF Blog: Bloomberg × OTel

First deck (architecture & internals):

`otel-collector-deep-dive/presentation.md`